CSS4003 — DevOps 2

Lecture 8: AWS Networking

Nauruzbayeva Farikha · Lesson 8

Recap — Lesson 7 (AWS EC2)

- What is an AMI, and why not hard-code its ID? (the image an instance boots from; IDs differ per region and change with updates, so resolve the latest from SSM)
- When does user data run? (once, on the first boot, through cloud-init — check `/var/log/cloud-init-output.log` when it fails)
- A security group has no rule for port 22. What happens to SSH? (it's blocked — inbound is deny-by-default, SGs only have allow rules)
- You stop and start an instance. What changes? (the auto-assigned public IP; the private IP and the root EBS disk stay)
- Why does user data ask for a token before calling `169.254.169.254`? (IMDSv2 — a session token protects metadata from SSRF-style requests)

Today’s agenda

- ☐ VPCs and CIDR blocks: how big is `10.0.0.0/16`?
- ☐ Subnets, and why each one lives in a single AZ
- ☐ Route tables, the internet gateway, and what makes a subnet "public"
- ☐ Private subnets, NAT gateways and Elastic IPs
- ☐ Security groups vs network ACLs
- ☐ Default VPC vs your own VPC, and a two-tier design
- ☐ S3 gateway endpoints, DNS, and a troubleshooting checklist

The scenario

Your web server from last week works. Now the team adds an app server with the database credentials on it. Someone launches it into the default VPC, where every subnet is public, and it gets a public IP. Its security group allows 22 "from the office" — which turns out to be `0.0.0.0/0`.

Today: design a network where only the web tier faces the internet, and the app tier can't be reached from outside at all.

Block 1

VPCs and CIDR

A VPC is your private network in a region

- **VPC** = Virtual Private Cloud: an isolated network that you define, inside one **region**.
- It spans **all the AZs** of that region. Subnets inside it pick an AZ.
- You choose its address range as a **CIDR block**, e.g. `10.0.0.0/16`.
- Traffic between VPCs, or to the internet, only flows where you build a path: gateways, routes, peering.
- EC2 instances, RDS databases and load balancers all live **in subnets of a VPC**.

CIDR in one slide

- `10.0.0.0/16` : the first **16 bits** are fixed (`10.0`), the rest is yours → 2^{16} = **65,536** addresses.
- `10.0.1.0/24` : 24 bits fixed → 2^{8} = **256** addresses (`10.0.1.0` – `10.0.1.255`).
- Each extra bit halves the block: `/25` = 128, `/26` = 64, `/28` = 16.
- A VPC block can be from `/16` (largest) to `/28` (smallest).

Rule of thumb: **VPC =** `/16` **, subnet =** `/24` . That gives up to 256 subnets of 256 addresses each — plenty for this course.

Use private address ranges

Range (RFC 1918)	Size
`10.0.0.0/8`	16.7 million addresses
`172.16.0.0/12`	~1 million (the default VPC uses `172.31.0.0/16`)
`192.168.0.0/16`	65,536 (common on home routers)

- These ranges are never routed on the public internet, so they're safe for internal networks.
- Pick a block that **doesn't overlap** your other VPCs or your office network — overlapping ranges can't be connected later.

AWS keeps 5 addresses in every subnet

For `10.0.1.0/24`:

Address	Reserved for
`10.0.1.0`	Network address
`10.0.1.1`	VPC router
`10.0.1.2`	Amazon DNS server
`10.0.1.3`	Reserved for future use
`10.0.1.255`	Broadcast (not supported, still reserved)

So a `/24` gives **251** usable addresses, and the smallest subnet, `/28`, gives 11.

Quick check

How many usable instance addresses does a `10.0.2.0/26` subnet have?

`/26` = 2^6 = 64 addresses, minus the 5 AWS reserves = **59**.

Block 2

Subnets, routes and the internet gateway

Subnets live in one AZ

- A **subnet** is a slice of the VPC's CIDR, e.g. `10.0.1.0/24`.
- Each subnet is in exactly **one Availability Zone**. It can't span two.
- High availability = the same tier in **two subnets in two AZs**.
- Every instance gets its private IP from its subnet's range.

```
VPC 10.0.0.0/16 (us-east-1)
├── 10.0.1.0/24  public   us-east-1a
├── 10.0.2.0/24  private  us-east-1a
├── 10.0.3.0/24  public   us-east-1b
└── 10.0.4.0/24  private  us-east-1b
```

Route tables decide where packets go

- Every subnet is associated with **one route table**. Subnets without an explicit association use the VPC's **main** route table.
- Every route table has a **local** route for the VPC's CIDR. You can't delete it — all subnets in a VPC can reach each other.
- More specific routes win: `10.0.0.0/16` beats `0.0.0.0/0`.

Destination	Target	Meaning
`10.0.0.0/16`	`local`	Stay inside the VPC
`0.0.0.0/0`	`igw-0abc...`	Everything else → internet gateway

The internet gateway (IGW)

- An **IGW** connects a VPC to the internet. One per VPC, managed by AWS, scales automatically, no bandwidth limit to plan for.
- Attaching it does nothing on its own. Traffic only uses it when a route table sends `0.0.0.0/0` to it.
- For an instance to be reachable from the internet it needs **all three**:
  1. its subnet's route table has `0.0.0.0/0 → igw`
  2. it has a **public IP** (auto-assigned or Elastic IP)
  3. its **security group** allows the port

What makes a subnet "public"?

Public subnet

- Route table has `0.0.0.0/0 → igw-…`
- Usually auto-assigns public IPs (`MapPublicIpOnLaunch = true`)
- For: web servers, load balancers, bastion hosts, NAT gateways

Private subnet

- Route table has **only** `local` (or `0.0.0.0/0 → nat-…`)
- No public IPs
- For: app servers, databases, anything that shouldn’t be reached from outside

"Public" isn't a checkbox on the subnet. It's **the route to the internet gateway**. Change the route table, and the same subnet becomes private.

Building it from the CLI

```
VPC=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query Vpc.VpcId --output text)
PUB=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.1.0/24 \
  --availability-zone us-east-1a --query Subnet.SubnetId --output text)
IGW=$(aws ec2 create-internet-gateway \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
RT=$(aws ec2 create-route-table --vpc-id $VPC --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id $RT --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --route-table-id $RT --subnet-id $PUB
```

Per the Learner Lab Readme, you can create VPCs in Learner Lab — in `us-east-1`. That's exactly today's practice.

Quick check

An instance has a public IP and a security group allowing port 80, but the page never loads. Its subnet's route table has only the `local` route. Why?

No `0.0.0.0/0 → igw` route: the subnet is private, so replies have no way out to the internet. A public IP alone isn't enough.

Block 3

Private subnets, NAT and Elastic IPs

Private instances that need to go *out*

- An app server in a private subnet sometimes needs outbound internet: package updates, calling an external API.
- It must still be **unreachable from the internet**.
- Solution: a **NAT gateway** in a **public** subnet, and in the private route table: `0.0.0.0/0 → nat-…`.
- NAT = network address translation: outgoing packets leave with the NAT gateway's public IP. Replies come back through it.
- Connections **started from the internet** have nowhere to go — there's no route in.

NAT gateway: what to know

- A public NAT gateway sits in a **public subnet** and uses an **Elastic IP**.
- It lives in **one AZ**. For high availability: one NAT gateway per AZ.
- It's billed **per hour** while it exists, **plus per GB** processed — even when nothing uses it.
- A common surprise on a student bill: a forgotten NAT gateway.
- Today's practice skips NAT: the private instance gets **no** internet access at all, which is enough to show the idea and costs nothing extra.

Check current prices on the AWS VPC pricing page before you create one.

Elastic IP, recap

- A **static** public IPv4 address you allocate to your account, then associate with an instance or a NAT gateway.
- Survives stop/start; can be moved to another instance in seconds.
- Every public IPv4 address is billed — attached or not. **Release** the ones you don't use.

```
EIP=$(aws ec2 allocate-address --query AllocationId --output text)
aws ec2 associate-address --allocation-id $EIP --instance-id $IID
aws ec2 release-address --allocation-id $EIP     # after disassociating
```

Block 4

Security groups vs network ACLs

Security groups, revisited

- Attached to an instance's **network interface**.
- **Stateful**: allow the request in, and the reply goes out automatically.
- **Allow rules only**; everything not allowed is denied.
- A rule's source can be a CIDR **or another security group**:

```
# app servers accept SSH only from instances in the web security group
aws ec2 authorize-security-group-ingress --group-id $SG_APP \
  --protocol tcp --port 22 --source-group $SG_WEB
```

Referencing a group instead of an IP keeps working when instances are replaced and get new IPs — the rule follows **membership**, not addresses.

Network ACLs: a second, subnet-level firewall

- Attached to a **subnet**; applies to every instance in it.
- **Stateless**: return traffic must be allowed **explicitly**.
- Has **allow and deny** rules, with numbers. Evaluated **lowest number first**; the first match wins.
- The VPC's **default NACL allows everything**. A new custom NACL **denies everything** until you add rules.
- Replies go to the client's **ephemeral port** (typically 1024–65535), so an outbound rule for those ports is needed.

Security group vs NACL

	Security group	Network ACL
Attached to	Instance (network interface)	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Evaluation	All rules together	In number order, first match
Default	Inbound denied, outbound allowed	Default NACL: all allowed
Typical use	Your main firewall	Blocking an IP range for a whole subnet

In this course you'll mostly use security groups and leave the default NACL alone. Know NACLs exist when a connection fails "for no reason".

Quick check

You add a custom NACL allowing inbound TCP 80. The web page still times out. The security group is fine. What's missing?

NACLs are stateless: you also need an **outbound** rule allowing the replies to the client's ephemeral ports (1024–65535).

Block 5

Designing a VPC

Default VPC vs your own

Default VPC

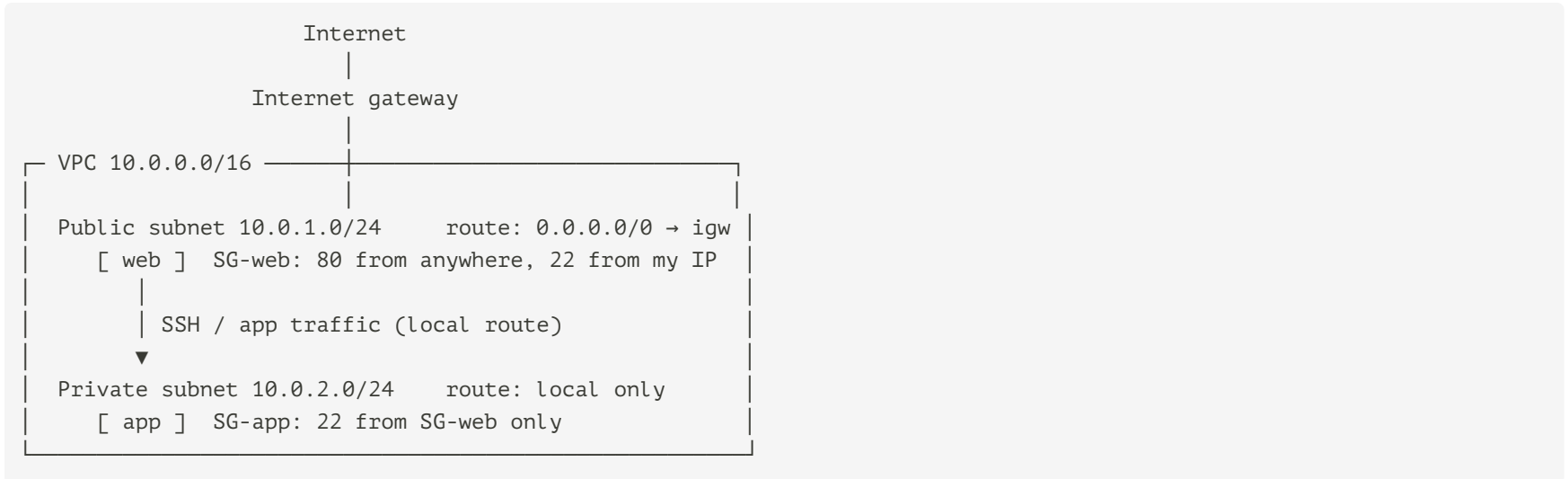
- Created by AWS in every region
- `172.31.0.0/16`, one `/20` subnet per AZ
- **All subnets public**, public IPs auto-assigned, IGW attached
- Great for quick tests (week 7)

Custom VPC

- You choose the CIDR and subnets
- Public **and** private subnets
- You control every route
- What real projects use

Putting a database or app server in the default VPC means it lands in a **public** subnet. One wrong security group rule and it's on the internet.

A two-tier design



This is exactly what you build in today's practice.

Reaching a private instance: the bastion

- The app server has **no public IP**. You reach it **through** the web server ("bastion" or "jump host").
- Keep your key on your laptop and forward the agent:

```
ssh-add labsuser.pem                       # load the key into your SSH agent
ssh -A -i labsuser.pem ec2-user@<web-public-ip>
# on the web server:
ssh ec2-user@10.0.2.x                      # private IP of the app server
```

In production, **SSM Session Manager** removes the bastion entirely: no open SSH port, no keys to copy.

VPC endpoints: reach S3 without the internet

- A private instance with no NAT can't reach S3 — S3's public endpoints are on the internet.
- A **gateway endpoint** for S3 adds a route to the route table: "S3's address ranges → endpoint". Traffic stays on the AWS network.
- Gateway endpoints exist for **S3** and **DynamoDB**, and have no extra charge.
- Other services use **interface endpoints** (PrivateLink), which are billed.

```
aws ec2 create-vpc-endpoint --vpc-id $VPC \
  --service-name com.amazonaws.us-east-1.s3 --route-table-ids $PRIV_RT
```

DNS inside a VPC

- Every VPC has the **Amazon DNS server** at the network base + 2 (e.g. `10.0.0.2`).
- `enableDnsSupport`: instances can use it (on by default).
- `enableDnsHostnames`: instances with public IPs get public DNS names like `ec2-3-80-...compute-1.amazonaws.com`. **On** in the default VPC, **off** in a VPC you create — turn it on:

```
aws ec2 modify-vpc-attribute --vpc-id $VPC --enable-dns-hostnames Value=true
```

Block 6

Troubleshooting and wrap-up

"I can’t connect" — check in this order

1. **Instance**: running? Status checks 2/2?
2. **Public IP**: does it have one? Is it the *current* one?
3. **Route table**: does its subnet have `0.0.0.0/0 → igw`?
4. **IGW**: attached to *this* VPC?
5. **Security group**: port and source allowed? Your IP changed?
6. **NACL**: default, or a custom one missing a rule (both directions)?
7. **OS**: is the service listening (`ss -tlnp`)? A local firewall?

> **Timed out** usually means steps 2–6 (packets dropped). **Connection refused** means you reached the machine but nothing listens — step 7.

Quick check

From the web server, `ssh` to the app server's private IP times out. Both are in the same VPC. Name the most likely cause.

SG-app doesn't allow port 22 from SG-web (or from the web server's address). The local route always exists, so routing isn't the problem.

Try it yourself

AWS Academy Learner Lab (`us-east-1`)

1. VPC `10.0.0.0/16` , a public and a private `/24`
2. IGW + public route table
3. SG-web and SG-app (22 only from SG-web)
4. Web instance in the public subnet, app instance in the private one
5. SSH to web, then hop to app
6. Remove the `0.0.0.0/0` route and watch it break
7. Delete everything, then **End Lab**

Without AWS (Linux VM or WSL)

Same shape with **network namespaces**:

```
sudo ip netns add web
sudo ip netns add app
sudo ip link add br-pub type bridge
sudo ip link add br-priv type bridge
# veth pairs = network interfaces,
# ip route   = route tables,
# iptables   = security groups
```

By the end of this lesson, you should be able to

- ☐ Size a VPC and its subnets with CIDR, and count usable addresses
- ☐ Explain what makes a subnet public or private
- ☐ Build a VPC, subnets, an IGW and a route table from the CLI
- ☐ Say when you need a NAT gateway, and why to delete it after use
- ☐ Compare security groups and NACLs, and use SG-to-SG rules
- ☐ Design a two-tier VPC and reach a private instance through a bastion
- ☐ Debug "can’t connect" with a step-by-step checklist

Before next lecture

- ☐ Delete everything from the practice — VPC last — and **End Lab**
- ☐ Make sure no NAT gateway or Elastic IP is left in your account
- ☐ Look at the S3 console: what does a bucket name look like?

Next week we add storage and a managed database to this network.

Next lecture

AWS Storage & Databases — EBS, S3 and RDS, and where they fit in the VPC you built today.